

Paws Rescue Donations

Project Brief

Prepared for Salman Adnan — reference document, not published content

Compiled July 2026

This is the concise companion to the full Deep Dive document. It covers the same project accurately but at a higher level, with a glossary at the end instead of inline definitions. Everything below was verified directly against the project’s source code and README – nothing is invented.

1 What It Is

Paws Rescue Donations is a small Django 4.2 web application for tracking donations at a pet rescue. It was built as a faculty-supervised community-service (“Khidmat”) project at Habib University. There is no public-facing side: every page except the login screen requires an authenticated staff (admin) account.

Staff log in with an email and password (no usernames, no public signup), record donations against individual pets, filter and sort the records, watch running totals on a dashboard, and export the currently filtered records to CSV. Admins can also create and remove other admin accounts from the web UI.

Why it exists (from the project’s own README): small rescues usually track donations in a spreadsheet, which breaks down once several people are entering records – there’s no access control, no consistent fields, and no easy way to answer “how much has Bella received this month?” This app replaces the spreadsheet with a single database and a login.

2 Architecture

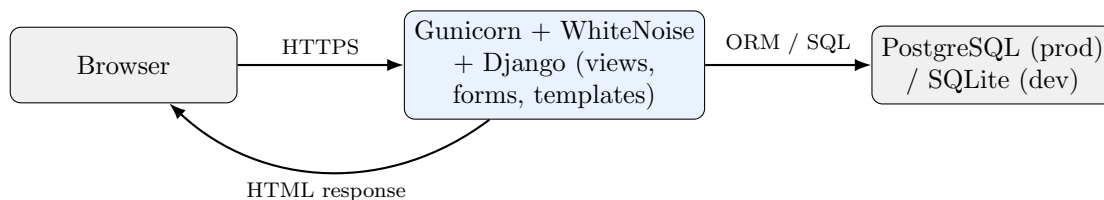


Figure 1: One Gunicorn process serves everything: HTML pages rendered by Django, static CSS served in-process by WhiteNoise, and data read from/written to whichever database `DATABASE_URL` points at (PostgreSQL in production, local SQLite in development).

Two Django “apps” live under `apps/`: **accounts** (custom email-based user model, login/logout, admin management) and **donations** (the `Donation` model, dashboard with filters and analytics, CRUD, CSV export). Everything is server-rendered with Django templates on a Bootstrap 5 layout.

Configuration comes from environment variables (via `python-dotenv`); a `Procfile` runs Gunicorn for deployment on a platform like Render or Railway.

3 Data Model

User (table: users)

```

PK id : UUID
  email : unique
  is_staff, is_superuser, is_active : bool
  created_at, updated_at

```

No relationship between these two tables –
no foreign key linking a donation to the admin who entered it.

Donation (table: donations)

```

PK id : auto-increment
  donor_name, donor_phone, donor_email
  pet_name
  amount : Decimal ≥ 0, currency (free text)
  payment_method : cash / bank / online / other
  reference_no, donation_date, notes

```

Figure 2: Both models were confirmed to be fully independent: no foreign key, one-to-one, or many-to-many field connects `Donation` to `User` anywhere in the codebase. This means there is no audit trail of who entered a given donation.

4 Key Features

- Email/password login with a custom user model; no public self-registration.
- Donation records with donor contact info, pet name, amount, currency, payment method, reference number, and notes.
- Filtering by donor/pet name, date range, payment method, currency, and amount range; sorting by date or amount; 20 rows per page.
- Dashboard analytics: all-time total, filtered total, and top-5-pets by donation amount within the current filters.
- CSV export of whatever the current filters show.
- Admin management screen (create admins, delete any admin except yourself).
- A one-time `create_initial_admin` management command that bootstraps the first login from environment variables.

5 Configuration and Deployment

Settings load from environment variables via `python-dotenv`. With `DATABASE_URL` set, the app uses PostgreSQL (via `dj-database-url` and `psycopg2`); left empty, it falls back to a local SQLite

file. Static files (just one CSS file; Bootstrap itself loads from a CDN) are served in-process by WhiteNoise, using a plain storage backend during test runs and a hashed/compressed manifest backend otherwise – this split is controlled by a `TESTING` flag (`'test' in sys.argv`) so that Django’s test runner, which never runs `collectstatic`, doesn’t crash on the production storage backend. The same `TESTING` flag also skips the production-only security settings (forced HTTPS, secure cookies, clickjacking protection), because those would otherwise turn every plain-HTTP test request into a redirect.

Deployment target: a Gunicorn-based host (Render, Railway, or similar) with a managed PostgreSQL database, started via the Procfile (`gunicorn paws_rescue.wsgi`). Required environment variables: `SECRET_KEY`, `DEBUG=False`, `ALLOWED_HOSTS`, `DATABASE_URL`, `INITIAL_ADMIN_EMAIL`, and `INITIAL_ADMIN_PASSWORD`.

6 Testing

Eleven automated tests (`python manage.py test`) cover: auth redirects, admin creation and self-deletion protection, donation CRUD, filtering by name and payment method, negative-amount validation, and the CSV export response.

7 Known Limitations (self-documented by the project)

These are drawn directly from the project’s own README (“Challenges” / “What I’d do differently”), not invented for this brief:

- CSV export reconstructs the “currently filtered” result set by stashing filtered donation IDs in the user’s session on every dashboard load, because the export link is a plain GET with no filter state of its own. The README calls this “the weakest part of the design” – it can grow unbounded and go stale between page load and export click. The stated fix would be to re-derive the filtered queryset from the query string inside the export view instead.
- No foreign key from `Donation` to `User`: no record of who entered a donation, and deletes are hard deletes with no history.
- `currency` is free-text with no validation, so inconsistent casing or typos could silently split totals that should be combined.
- No database indexes on `donation_date` or `pet_name`, despite both driving the default sort order and the top-pets analytics.
- Filtering logic is duplicated field-by-field with no shared helper between the dashboard and export views.

Glossary

App

A self-contained module of a Django project; this project has two, `accounts` and `donations`.

CRUD

Create, Read, Update, Delete – the four basic data operations a management screen typically provides.

CSRF

Cross-Site Request Forgery; Django blocks it with a secret per-session token embedded in every form.

CSV

Comma-Separated Values, a plain-text spreadsheet file format.

Foreign key

A column linking one table's row to another table's row – absent between `Donation` and `User` here.

Framework

Pre-built application scaffolding; Django supplies the web server glue, database layer, and templating so the developer only writes the app-specific parts.

Gunicorn

The production-grade Python web server that actually runs the app once deployed (as opposed to Django's own development-only server).

Management command

A custom `manage.py` subcommand – here, `create_initial_admin`.

Migration

A versioned, incremental file describing one database schema change; Django applies pending ones with `manage.py migrate`.

Model

A Python class that Django maps onto a database table.

MVT

Django's Model-View-Template pattern: Model = data/database layer, View = the Python logic that decides what to show (plays the role classic MVC calls the "Controller"), Template = the HTML that gets filled in and rendered.

ORM

Object-Relational Mapper – lets Python code build database queries by calling methods on model classes instead of writing raw SQL by hand.

Session

Server-side, per-browser data storage keyed by a cookie.

UUID

A 128-bit identifier practically guaranteed to be unique; used as the primary key on the `User` model instead of a simple counter.

WhiteNoise

A library that lets the Django/Gunicorn process serve its own static files (CSS) directly, without a separate web server in front of it.

WSGI

The standard interface between a Python web server (Gunicorn) and a Python web framework (Django).